

RETO 1 - SEMANA 1 | Primer Parcial

Unidad 1: Fundamentos y Drivers Arquitectónicos

Proyecto IoT: Seguridad perimetral con IA básica

Programa / curso	Arquitectura de Software
Integrante(s)	Jose Barrios – Fabian Alvarez

Desafío abordado: definir los cimientos de un sistema IoT de seguridad perimetral: necesidades, componentes, estilos arquitectónicos y validación mínima con MQTT.

1. Descripción del proyecto

El proyecto propuesto consiste en un sistema IoT de seguridad perimetral con IA básica para viviendas, locales pequeños o zonas restringidas. El sistema utiliza un sensor PIR para detectar movimiento, una cámara ESP32-CAM para capturar evidencia y MQTT como mecanismo de comunicación entre dispositivos y los componentes de procesamiento.

Cuando se detecta un evento, el nodo de borde publica el mensaje al broker MQTT. Luego, Node-RED procesa la información, aplica reglas y una validación simple de IA básica para reducir falsos positivos, registra la evidencia y genera una alerta hacia un panel de monitoreo y un canal de notificación.

Como el enunciado original solo menciona PIR, cámara ESP32, MQTT y Node-RED, se asume una solución inicial de bajo costo, fácil de desplegar y preparada para crecer hacia más sensores, más zonas y reglas de análisis más avanzadas.

Objetivo arquitectónico

Construir una solución confiable, segura, modular y escalable que detecte intrusiones en tiempo real y entregue alertas útiles con evidencia, sin depender de una infraestructura costosa.

2. Drivers arquitectónicos

A continuación se identifican ocho drivers arquitectónicos (mínimo requerido: seis) para el proyecto.

ID	Tipo	Necesidad / restricción	Implicación arquitectónica	Prioridad
D1	Seguridad	Evitar accesos no autorizados al broker, dashboard y evidencias.	Uso de autenticación en MQTT, credenciales por dispositivo, control de acceso por roles y cifrado cuando el entorno lo	Alta

ID	Tipo	Necesidad / restricción	Implicación arquitectónica	Prioridad
			permita.	
D2	Disponibilidad	Mantener vigilancia y registro de eventos aun ante reinicios o microcortes.	Broker estable, reintentos, retención de mensajes críticos y tolerancia a fallos parciales.	Alta
D3	Rendimiento / Latencia	La alerta debe llegar rápido para que tenga valor operativo.	Pipeline orientado a eventos, procesamiento liviano y envío inmediato al dashboard/notificación.	Alta
D4	Escalabilidad	Permitir agregar nuevas zonas, sensores o cámaras sin rediseñar todo el sistema.	Tópicos MQTT por zona/dispositivo, componentes desacoplados y reglas reutilizables.	Media-Alta
D5	Modificabilidad	Facilitar cambios futuros: agregar sensores magnéticos, sirena, almacenamiento o nuevas reglas de IA.	Separación en capas y flujos modulares en Node-RED; contratos de mensajes JSON consistentes.	Alta
D6	Trazabilidad	Cada evento debe poder auditarse y revisarse posteriormente.	Registro de fecha, dispositivo, zona, evidencia y estado de la alerta en base de datos/bitácora.	Media-Alta
D7	Costo / Viabilidad	La solución debe ser implementable con hardware accesible y curva de aprendizaje razonable.	Selección de ESP32-CAM, PIR, MQTT y Node-RED por bajo costo y madurez.	Media
D8	Usabilidad operativa	El usuario debe entender qué ocurrió, dónde ocurrió y qué acción tomar.	Dashboard claro, nombres de zonas legibles, estados y notificaciones con contexto.	Media

3. Escenarios de calidad formales

Se presentan tres escenarios redactados con la estructura solicitada: fuente -> estímulo -> artefacto -> entorno -> respuesta -> medida.

EC-1 - Seguridad	
Fuente	Usuario malicioso o dispositivo no registrado.
Estímulo	Intenta publicar mensajes en un tópico MQTT o ingresar al dashboard sin autorización.
Artefacto	Broker MQTT y módulo de autenticación/autorización del dashboard.
Entorno	Operación normal del sistema, con vigilancia activa.
Respuesta	El sistema rechaza la conexión o la publicación, registra el intento y mantiene el servicio disponible para los usuarios autorizados.
Medida	100% de conexiones no autorizadas rechazadas; registro del intento en menos de 5 segundos; sin caída del servicio.

EC-2 - Disponibilidad	
Fuente	Falla de red Wi-Fi, reinicio del broker o microcorte eléctrico.
Estímulo	Se pierde la conectividad mientras un sensor detecta movimiento.
Artefacto	Nodo Edge, broker MQTT y flujo de procesamiento en Node-RED.
Entorno	Horario de monitoreo activo y presencia de eventos reales.
Respuesta	El nodo intenta reenviar el evento cuando vuelve la conexión; el broker se recupera y Node-RED reanuda el consumo sin reconfiguración manual extensa.
Medida	Recuperación operativa en menos de 60 segundos; al menos 95% de los eventos críticos reenviados tras reconexión; pérdida máxima aceptable de 1 evento no crítico por incidente.

EC-3 - Modificabilidad	
Fuente	Equipo de desarrollo o administrador técnico.
Estímulo	Se solicita agregar una nueva zona con un sensor magnético de puerta, sin afectar la lógica actual de movimiento.
Artefacto	Modelo de tópicos MQTT, flujos Node-RED, dashboard y persistencia.
Entorno	Sistema en operación con una o más zonas ya

	desplegadas.
Respuesta	Se crea un nuevo tópico, se reutiliza el flujo parametrizado, se añade la nueva zona al dashboard y se extiende el esquema de eventos sin reescribir el sistema completo.
Medida	Cambio implementado en menos de 4 horas de trabajo; sin modificar más del 20% de los flujos existentes; sin interrupción total del monitoreo.

4. Diagramas C4: contexto y contenedores

Los siguientes diagramas representan los niveles 1 y 2 del modelo C4. También se adjuntan versiones en formato .drawio para edición en Draw.io.

Figura 1. C4 - Nivel 1 (Contexto)

Diagrama C4 - Nivel 1 (Contexto)

Proyecto IoT: Seguridad perimetral con IA básica

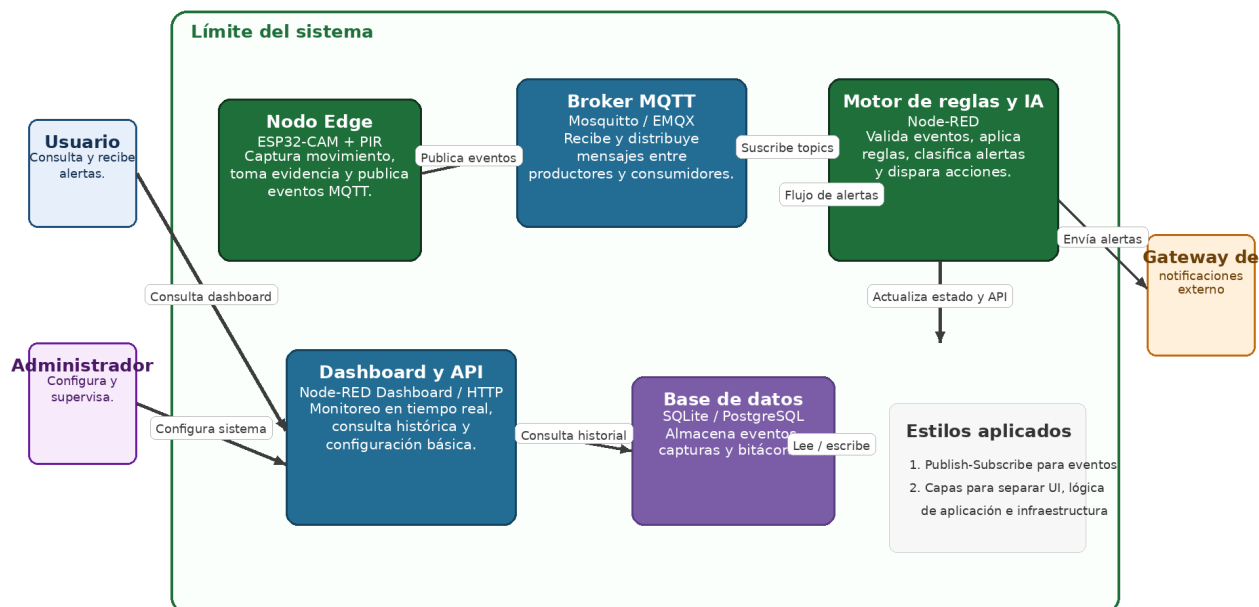


Interpretación: el sistema se relaciona con dos actores principales (usuario y administrador) y con un servicio externo de notificaciones. El corazón del sistema es la plataforma IoT que concentra la detección, el procesamiento de eventos, el monitoreo y la evidencia.

Figura 2. C4 - Nivel 2 (Contenedores)

Diagrama C4 - Nivel 2 (Contenedores)

Sistema IoT: Seguridad perimetral con IA básica



Interpretación: la solución se divide en cinco contenedores clave: nodo Edge, broker MQTT, motor de reglas/IA en Node-RED, dashboard/API y base de datos. Esta organización permite separar adquisición, mensajería, procesamiento, visualización y persistencia.

5. Justificación de dos estilos arquitectónicos

Estilo 1: Publish-Subscribe (Pub-Sub)

- Es el estilo principal del sistema porque los sensores generan eventos asíncronos y no deben depender directamente de quién los consume.
- MQTT implementa este estilo de manera natural: el nodo ESP32 publica en tópicos y múltiples consumidores pueden suscribirse al mismo evento sin acoplarse al emisor.
- Este enfoque mejora escalabilidad, flexibilidad y tolerancia al cambio. Por ejemplo, una misma detección puede ser consumida por Node-RED para alertar, por la base de datos para registrar y por un módulo futuro de analítica sin tocar el firmware del sensor.
- Además, Pub-Sub es adecuado para IoT por su bajo consumo de ancho de banda y su operación eficiente sobre redes inestables.

Estilo 2: Arquitectura en capas

- Aunque el flujo de eventos se mueve con Pub-Sub, la aplicación necesita organización interna clara. Por eso se adopta una arquitectura en capas.
- Capa de presentación: dashboard y vistas de monitoreo para usuario y administrador.
- Capa de aplicación / lógica: Node-RED orquesta reglas, validación de IA básica, notificaciones y coordinación de casos de uso.
- Capa de integración: clientes MQTT, conectores de correo/Telegram y adaptadores hacia hardware.
- Capa de persistencia: base de datos y bitácora de evidencias.
- Este estilo reduce el acoplamiento, facilita pruebas, mejora mantenibilidad y hace más sencillo evolucionar la solución a futuro.

6. Prueba MQTT mínima

Objetivo: demostrar publicación y suscripción de un dato real del sistema IoT utilizando MQTT. La prueba puede ejecutarse con un broker local (por ejemplo Mosquitto), el nodo ESP32 o una publicación manual desde terminal.

Topología mínima de prueba

- Publicador: ESP32-CAM o terminal que simula el sensor.
- Broker: Mosquitto.
- Suscriptor: Node-RED Debug / Dashboard y un terminal con mosquitto_sub.
- Tópico sugerido: casa/perimetro/zona1/movimiento.

Payload sugerido

```
{"deviceId":"esp32cam-z1","zone":"zona1","motion":true,"confidence":0.83,"timestamp":"2026-03-23T20:30:00-05:00"}
```

Comandos de validación

```
Suscripción: mosquitto_sub -h localhost -t "casa/perimetro/zona1/movimiento" -v
```

```
Publicación: mosquitto_pub -h localhost -t "casa/perimetro/zona1/movimiento" -m '{"deviceId":"esp32cam-z1","zone":"zona1","motion":true,"confidence":0.83,"timestamp":"2026-03-23T20:30:00-05:00"}'
```

Resultado esperado

- El terminal suscriptor recibe el payload inmediatamente después de la publicación.
- Node-RED muestra el evento en el panel o en el nodo Debug.
- La alerta se guarda en la bitácora y se puede asociar a la zona de origen.
- Tiempo objetivo de punta a punta: hasta 2 segundos en red local para visualización, y hasta 5 segundos para notificación.

8. Conclusión

La propuesta arquitectónica cumple con los elementos solicitados en la actividad: drivers, escenarios de calidad, diagramas C4, estilos arquitectónicos, prueba mínima MQTT y guion de pitch.

Además, deja una base sólida para continuar con el desarrollo del prototipo, porque organiza el sistema alrededor de decisiones realistas para IoT: comunicación desacoplada, bajo costo de hardware, monitoreo centralizado y crecimiento gradual.